

Module Ten Journal: Reflections on Advanced Programming Language Theory

Aaron Foster

Southern New Hampshire University

CS 660: Advanced Programming Language Theory

Prof. Shadha Tabatabai

September 7th, 2026

The Most Challenging Work

The hardest work in this course was the Module Seven language comparison, and the difficulty was not writing code. It was making the measurement mean something. Comparing Python and Go on sorting is easy to do badly: different algorithms, different input, a cold interpreter, a single timed run. I treated it as an experiment rather than a demonstration. The three sorts were ported line for line, both languages received identical input, every figure was the mean of five iterations after an untimed warm-up, and a C implementation compiled to a shared library served as a fixed baseline. What mattered most was fixing the methodology before seeing any numbers, so I could not tune the setup toward the conclusion I already expected.

A Concept That Reshaped My Thinking

The Module One study of Pascal changed my thinking more than any single technical topic. By its designer's standards Pascal succeeded: small, strongly typed, teachable. It still lost. Arrays carried their length in their type, so no routine could accept a string of arbitrary size, while C spread through universities essentially free with Unix. What reshaped my thinking is that the ideas did not die with the language. Strong static typing and structured design won completely, and I have used both in every language this term. Design merit and survival are different variables, and the second depends heavily on distribution, tooling, and timing. I now read a language's popularity as evidence about its ecosystem rather than about its quality.

Paradigms and Languages as Problem-Solving Tools

Writing one algorithm three ways in Module Five made the differences concrete. The imperative version forced me to invent a result list the problem never mentioned; the functional version carried no such state, and the problem's two rules became named, testable predicates; the object-oriented version was the least concise and the most extensible. None was better in general

— each was better at something. That framing decided the Module Nine proposal. The legacy system's bottleneck was a single global lock, and procedural code offers no construct binding an invariant to the data it protects, so the discipline lived in the programmer's memory. Objects let each inventory item own its quantity and its own lock, which is what made per-item concurrency expressible at all. The paradigm was not a matter of style; it was what made the fix possible.

Applying This to Future Coursework

With coursework remaining in this program, the habit I intend to carry forward is justifying a technical choice rather than defaulting to one. The Module Nine project required recommending Java over Python for a legacy rewrite, and the recommendation held only because it was tied to a stated requirement — 500 transactions per second at a 75 ms tail — instead of to preference. I plan to state the requirement first, evaluate the candidates against it, and name what would change my answer. Writing an honest tradeoff section is part of that: the costs I listed against my own recommendation are what made it credible.

How My Skill in Evaluating Languages Changed

My confidence changed in kind more than in degree. At the start I evaluated languages as a user, asking which one I would rather write in. I now ask why a language was built the way it is. Writing a parser in Module Two, a mark-sweep collector in Module Six, and a small domain-specific language in Module Eight moved grammar, memory management, and type systems from features I used to decisions someone made under constraints. The clearest evidence of the shift is that I am at once more willing to make a language recommendation and more careful about how broadly I state it, because I now know what a fair comparison costs to produce.